

Comparing Fuzz4All Performance with Newer Models

Eric Weng (qgt7zm)
May 7, 2026

I. INTRODUCTION

Fuzzing is a widely-used form of software testing used to discover security-critical bugs. Fuzzing can be performed on any program, as long as a compiler is available that can instrument the source code or binary executable for coverage. Software such as compilers and constraint solvers, which accept programming or formal languages, are of particular interest because the rest of software development relies on their correctness and security. These programs often take in very complex inputs, and fuzzing them requires deep domain expertise.

However, traditional fuzzers struggle to generate diverse, valid, and distinct test cases for compilers that traverse a meaningful number of paths. Random fuzzing mutations are unaware of language semantics and often created malformed programs that are quickly rejected. Crafting a good seed corpus takes time, and eventually the input space will become saturated as no more productive mutations can be performed.

Furthermore, as languages are frequently created (or updated) to solve problems in existing ones, or projects may use more than one language, developers may desire to be able to easily fuzz multiple compilers with minimal effort. However, the knowledge used to fuzz one language’s compiler is not easily transferrable to another language, a different language version, or even other language features.

Fortunately, the paper *Fuzz4All: Universal Fuzzing with Large Language Models* [4] attempts to address these exact problems by creating a universal fuzzer that uses an LLM to generate fuzzing inputs. Fuzz4All first summarizes natural language documentation using a *distillation* LLM. The distillation LLM also iteratively refines fuzzing prompts using the *autoprompting* technique. Then, Fuzz4All feeds the summary and prompt to a second *generation* LLM, which automatically synthesizes test cases for the target.

The goal of my project is to extend Fuzz4All to work with more recent LLMs and compare how different models affect the amount and growth of code coverage. In addition, I will analyze the generated inputs to determine why each model achieves the level of coverage that it did.

My results will be useful to security researchers interested in knowing which model to select for fuzzing and LLM developers wanting to create models specialized for fuzzing.

II. BACKGROUND

A. Fuzzing Challenges

Traditional fuzzers can be broadly divided into two categories: mutational and generational. Mutational fuzzers perform random changes to an input, such as adding, deleting, or moving bytes. Meanwhile, generational fuzzers procedurally create inputs following a grammar. Specific applications exist that create sample programs in a certain language, such as Csmith for C and YarpGen for C++.

Mutational fuzzers have trouble getting past “magic constants”—in this case keywords and other named constructs. Even generational fuzzers can only target a limited subset of the language’s grammar at a time. Additionally, if a language and its compiler evolve and the fuzzer is not updated, then any new features cannot be fuzzed at all.

To illustrate the difficulty of fuzzing multiple compilers with the same fuzzer, consider two simple “Hello, World” programs in Java and C:

```
// C Hello World
#include <stdio.h>

int main() {
    printf("%s", "Hello, world!");
    return 0;
}

// Java Hello World
public class Main {
    public static void main(String[]
        ↪ args) {
        System.out.println("Hello,
            ↪ world!");
    }
}
```

Even a programmer unfamiliar with these languages can still point out the numerous differences between syntax, keywords, function calls, and include statements between these two programs. It is highly unlikely that random mutations could transform the line `#include <stdio.h>` into `public class Main` consistently and in a reasonable amount of time. Moreover, while basic input and output could be trivially converted between languages, more advanced constructs, such as arrays, objects/structs, methods, and pointers (which Java does not have) would be much harder to translate.

B. Fuzz4All

Fuzz4All consists of an autoprompting stage and a fuzzing loop [4]. The autoprompter facilitates the task of prompt engineering, which may otherwise take humans several iterations to get right. The authors selected GPT-4, a powerful model with strong natural language processing capabilities for the distillation role. They were not concerned about the LLM cost since fuzzing makes up the bulk of the responses. For each language, they first fed GPT-4 documentation, including descriptions of functions and modules as well as example code, to produce condensed summaries. Next, during autoprompting, the authors had the distillation model generate several fuzzing prompts (e.g. “Please create a short program which uses new C features in a complex way”). Then, they evaluated each candidate over a few fuzzing runs and scored each prompt based on the resulting code. The authors chose to rank prompts by number of valid programs, but they mentioned other metrics could be coverage or program length).

During the fuzzing loop, Fuzz4All feeds the summary and best prompt to the fuzzing model, which generates as many inputs as possible. The authors selected StarCoder, a small, specialized model to minimize the inference overhead for each loop iteration. To make the most out of a single prompt, the authors devised multiple fuzzing strategies. The fuzzing model can either create a program from scratch or, given an example, mutate the existing program or generate one semantically equivalent to it. Lastly, input evaluation can be performed after fuzzing to collect coverage and run oracles (typically crash or hang).

The authors compared Fuzz4All against state-of-the-art compiler fuzzers, with said compilers serving as systems-under-test. On average, Fuzz4All covered 40% more lines of code than the baselines, and kept discovering coverage long after the traditional fuzzers had plateaued. Counting the number of inputs reveals that Fuzz4All creates both fewer total and valid programs. In short, Fuzz4All focuses on quality over quantity, with fewer “duplicate” inputs that traverse the same path.

Some improvements I would have made are listing the coverage as a percentage of total SUT lines of code, comparing the length of generated inputs, and analyzing the types of programs generated by Fuzz4All.

C. LLM Advancements

The model used in the paper, StarCoder, is from 2023 and may not reflect the most recent advancements in LLM technology [1]. Since the paper’s publication, reasoning models with chain-of-thought and self-critique models have become more popular, as have compact open-weight models [3]. Most notably, 2025, DeepSeek-R1 took the world by storm with its ability to outperform larger, commercial, flagship models such as GPT thanks to its training from synthesized data [2].

III. METHODS

In this project, I run Fuzz4All using with more up-to-date LLMs that I believe will generate better inputs. I seek to answer the following research questions:

- 1) Do more recent models result in better fuzzing coverage?
- 2) Do larger or smaller models result in greater coverage?
- 3) How do the length and content of programs affect coverage?

TABLE I
SELECTED FUZZ4ALL MODELS (* INDICATES BASELINE)

Name	Size	Hash	Year
starcoder:3b*	1.8 GB	847e5a7aa26f	2023
starcoder2:3b	1.7 GB	9f4ae0aff61e	2024
starcoder2:7b	4.0 GB	1550ab21b10d	2024
codegemma:2b	1.6 GB	926331004170	2024
codegemma:7b	5.0 GB	0c96700aaada	2024
qwen3.5:2b	2.7 GB	324d162be6ca	2026
qwen3.5:4b	3.4 GB	2a654d98e6fb	2026

For my experimental setup, I run the authors’ Docker container [5], with the source code and binaries provided, on the DADA Ubuntu VMs. While I attempted to use both the CS Portal Apptainer engine and SLURM compute clusters, a VM provides with better control over my development environment and local models. Since the DADA servers do not have GPUs, my selection of models is less ideal than I would have preferred. I ended up selecting the following Ollama open-source models (see Table I):

- StarCoder (baseline), a lightweight code generation model;
- StarCoder2, an updated version of StarCoder;
- CodeGemma, another coding model; and
- Qwen3.5: a general-purpose reasoning-model.

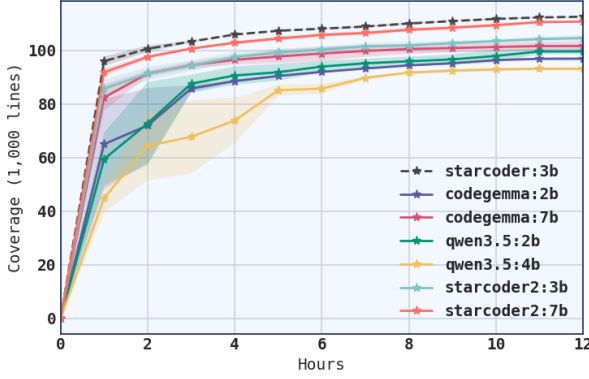


Fig. 1. Coverage over Time for gcc

I pick GCC and G++ as my targets, which compile C and C++ respectively. For each target and model, I run two fuzzing campaigns over 12 hours and collected coverage afterward. Since the authors’ results already included the best prompt, I skipped the autoprompting stage altogether. Each run uses the same fuzzing strategy, which is to generate a semantically equivalent program. Lastly, I recreate Figure 4 showing coverage over time and Table 2 showing the count and validity of programs.

Besides recreating the experiments, I perform a deeper analysis into the programs generated by each model. In Table 2, I also list the number of programs generated average program length for each model, allowing to me to judge speed and test case complexity. In addition, I inspect the first and last program (theoretically the least and most refined) of each run, viewing the contents and compilation messages.

I did not modify any of the source code, but I did create new utility programs to generate figures and run configurations supporting different models. I published my changes plus instructions to a GitHub repository: <https://github.com/qgt7zm/fuzz4all-project/>.

IV. RESULTS

Contrary to what I predicted, no newer models outperformed the baseline model (see Figures 1 and 2). The baseline StarCoder model had the best line coverage, while the Qwen3.5 models had the worst. The coverage range for g++ was much greater, at 110k–170k LoC vs 90k–110k LoC, likely owing to the massively increased complexity of C++ over C. The StarCoder2 models came closest to reaching the original. Larger models performed better than smaller models in four out of six cases, suggesting the smaller LLMs did not retain enough of their distilled summary, were distracted by the prompt. However, the difference was minor, at 5k–13k LoC.

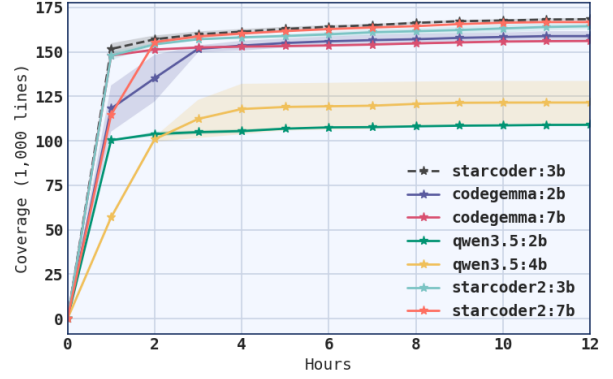


Fig. 2. Coverage over Time for g++

TABLE II
GENERATED PROGRAM METRICS FOR GCC

Model	Programs	Valid	Line Cov.	Avg. LoC
starcoder:3b	2113	75.08%	112,607	22
codegemma:2b	344	4.55%	96,958	109
codegemma:7b	299	49.40%	101,681	33
qwen3.5:2b	179	6.50%	99,675	88
qwen3.5:4b	43	7.01%	93,182	53
starcoder2:3b	1535	57.69%	104,637	36
starcoder2:7b	2663	79.74%	110,797	11

TABLE III
GENERATED PROGRAM METRICS FOR G++

Model	Programs	Valid	Line Cov.	Avg. LoC
starcoder:3b	2268	64.35%	168,363	23
codegemma:2b	421	3.75%	158,849	123
codegemma:7b	770	40.84%	156,084	13
qwen3.5:2b	70	0.00%	108,968	22
qwen3.5:4b	32	1.61%	121,498	20
starcoder2:3b	3244	67.06%	164,414	11
starcoder2:7b	3488	69.43%	166,734	9

By simply counting the programs and lines of code (LoC) outputted by each model (Tables II and III), we gain insight into how each model functions. The StarCoder2 model typically generated the most total and valid programs, which were usually very short (below 20 LoC). In contrast, the Qwen3.5 models generated the fewest programs, with an appalling 0-7% validity rate across all targets. Interestingly, CodeGemma (specifically with 2B parameters) generated the longest programs (well over 100 LoC), which were and mostly invalid but still more numerous than Qwen3.5.

When looking at the individual test cases, I noticed the same few mistakes occurred repeatedly. Models will often forget to include external headers to define types ahead of usage, which will cause the compiler to reject the rest of the code, even if it are no other errors. Another

tendency is for models to attempt to use C++-exclusive features when producing C code. Perhaps models confuse the two languages since they are often used together. Other times, models will produce incomplete programs, cutting off before the `main` function can be completed. All these errors can be considered hallucinations.

From my results, I obtained the following answers to my questions:

- 1) More recent models do not necessarily result in better fuzzing coverage.
- 2) Larger models generally result in slightly greater coverage at least up to 7B parameters.
- 3) The optimal test suite is composed of many short-to-medium-length programs.

V. CONCLUSIONS

Ultimately, I discovered that original StarCoder achieved the best coverage by generating numerous, mostly-valid programs of a moderate size (20 LoC). Shorter programs are less likely to contain mistakes, but longer programs are useful to exercise deeper paths with a single case. The successor, StarCoder2, produces more total test cases, but at the cost of also more duplicate or shallower ones. CodeGemma generates fewer, longer inputs, which are often riddled with errors. Qwen3.5’s reasoning became its greatest weakness, as it spent too much time deliberating to produce useful inputs in any useful quantity.

My greatest takeaway from this project is that coding is not the same as compiler fuzzing. While on the surface both activities entail generating programs, fuzzing is open-ended and repeated, whereas coding is directed and one-time. Fuzzing requires that the model generate any program over and over and not take too long with each iteration. In contrast, software engineering means solving a problem once under narrower constraints, allowing more time dedicated towards the solution.

I believe that the key to fuzzing with LLMs is to use specialized or fine-tuned models over the latest and greatest. Any future fuzzing model needs to balance program quantity and quality. Lightweight models trained mostly on code (or even one language if possible) would be least prone to hallucinating in other languages. Reasoning models are unsuitable for fuzzing because they infer too slowly to gain enough “critical” mass in total output. My research also indicates that off-the-shelf, open-source models are sufficient for security researchers. At the moment, larger models appear to generate more grounded and complex responses. Small models more likely to make mistakes or run out of memory by entering feedback loops (compare the probable results of studying one hour vs one day for an exam). However,

there is no need for gigantic trillion-parameter models requiring large data centers to run, meaning fuzzing can be performed with self-hosting.

The primary limitation of my project was the small scale. The paper writers had access to a 64-core workstation with 256 GB RAM and multiple dedicated GPUs. My DADA VMs had much fewer resources, which were potentially shared by other users. As a result, my conclusions may not be as thorough and generalizable as those in the original study. If I were to redo my experiments, I would ideally want a GPU to allow me to run larger 10–20 billion-parameter models (or simply run smaller models more efficiently). Testing at a larger scale would also better highlight the trends. Perhaps testing with models larger than 7 billion parameters could reveal that the marginal increase in coverage eventually drops to zero.

Some areas of future study would involve fuzzing with other languages and targets. Particularly, I want to see if models mix up other languages like they did C and C++. I would also modify the fuzzing prompt and strategy, as well as experiment with compiler options (something the Fuzz4All authors did not touch). To lend more support to the “universality” of Fuzz4All claimed by the researchers, I would explore other software where generating good seeds and mutants requires extensive domain knowledge. After all, the model is simply learning to create objects that follow a grammar. Lastly, I would study both the programs generated by traditional fuzzers and LLMs to discover if a fuzzing strategy exists that combines the best both of worlds. Overall, LLM-based automated fuzzing makes fuzzing complex programs quicker and cheaper and allows researchers to focus on the more important tasks of finding and triaging bugs.

REFERENCES

- [1] StarCoder: A State-of-the-Art LLM for Code. von werra, leandro and ben allal, loubna, 2023.
- [2] Wikipedia. Deepseek, 2026.
- [3] Wikipedia. Large language model, 2026.
- [4] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE ’24*, New York, NY, USA, 2024. Association for Computing Machinery.
- [5] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal Fuzzing with Large Language Models, January 2024.